

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3092 - Deep Learning y Sistemas Inteligentes

Luis Suriano



Laboratorio 8

Del Mini-GPT a BERT: muestreo autoregresivo y
embeddings contextuales

Ricardo Godínez - 23247

Vianka Castro - 23201

Diego Lopez - 23747

GUATEMALA, 3 de octubre de 2026

1. Introducción

En este laboratorio recorrimos las dos grandes familias de modelos basados en transformers desde dos lados distintos. En la entrega parcial construimos un Mini-GPT a nivel de caracteres (un modelo que lee y genera texto letra por letra) y lo entrenamos con Tiny Shakespeare, un corpus de poco más de un millón de caracteres con obras de Shakespeare. Con ese modelo entrenado comparamos tres formas de convertir sus predicciones en texto: greedy, top-k y top-p.

En la entrega final cambiamos de lado. En vez de entrenar, cargamos un BERT multilingüe ya preentrenado y lo usamos para extraer embeddings contextuales (vectores que representan una palabra tomando en cuenta la oración en la que aparece) de tres palabras con varios significados: banco, copa y vela. Los comparamos con los embeddings estáticos de Word2Vec, que asignan un único vector a cada palabra, y los visualizamos con PCA. Al final respondimos dos preguntas que conectan ambas partes: por qué GPT necesita la máscara causal y por qué el paradigma de preentrenar y adaptar nos deja elegir entre GPT y BERT según la tarea.

El Mini-GPT reutiliza las ideas del Proyecto 1 de la Semana 6, pero adaptado de palabras a caracteres, con batches, un contexto de 128 caracteres y tres bloques decoder apilados.

2. Objetivos

Objetivo general

Comprender cómo la máscara de atención define lo que un transformer puede hacer, entrenando un modelo autoregresivo desde cero y comparándolo con un modelo bidireccional preentrenado.

Objetivos específicos

- Implementar y entrenar un Mini-GPT a nivel de caracteres sobre Tiny Shakespeare y medir su perplejidad de validación.
- Implementar desde cero las estrategias de muestreo greedy, top-k y top-p, y analizar 25 generaciones en cinco configuraciones.
- Extraer con `bert-base-multilingual-cased` los embeddings de la última capa para palabras polisémicas en contextos distintos.
- Comparar la similitud coseno de esos embeddings con la de los vectores estáticos de Word2Vec.

- Visualizar los embeddings de BERT con PCA y discutir si los significados se separan.
- Argumentar, a partir de los resultados, el papel de la máscara causal y del paradigma de preentrenar y adaptar.

3. Desarrollo

Task 1.1: corpus, vocabulario y batches

Empezamos descargando Tiny Shakespeare y armando el vocabulario con los caracteres únicos del texto. Obtuvimos 1,115,394 caracteres en total y un vocabulario de 65 símbolos (letras mayúsculas y minúsculas, signos de puntuación, espacio y salto de línea). Codificamos toda la secuencia como enteros y la dividimos 90/10 respetando el orden del texto: 1,003,854 caracteres para entrenamiento y 111,540 para validación. No barajamos los caracteres porque eso habría mezclado pedazos de validación dentro del entrenamiento.

En cada batch el objetivo y es la misma entrada x corrida un carácter ($y_t = x_{t+1}$). Así, de un bloque de 128 caracteres salen 128 ejemplos de predicción a la vez. Cada época recorre todos los bloques completos de entrenamiento una vez; barajamos el orden de los bloques, pero nunca el orden interno de los caracteres.

Arquitectura del Mini-GPT

Del Proyecto 1 reutilizamos `layer_norm`, las proyecciones $Q = XW_Q$, $K = XW_K$ y $V = XW_V$, la atención $\text{softmax}(QK^T / \sqrt{d_k})V$, la máscara causal, los residuales y el feed-forward. Lo que tuvimos que cambiar fue la tokenización por carácter, los tensores con forma (batch, tiempo, dimensión), el contexto de 128, las cuatro cabezas de atención, los tres bloques apilados y una cabeza de lenguaje que produce 65 logits (puntajes sin normalizar, uno por carácter posible) en cada posición. La Tabla 1 resume los hiperparámetros, que corresponden a los mínimos del enunciado.

Tabla 1. Hiperparámetros del Mini-GPT.

Hiperparámetro	Valor
Contexto (block_size)	128
Dimensión del modelo (d_model)	128
Cabezas de atención	4
Bloques decoder	3

Hiperparámetro	Valor
Tamaño de batch	64
Tasa de aprendizaje	3e-4
Épocas	10
Parámetros totales	609,985

El corazón del bloque es la self-attention con máscara causal. Dejamos comentada en el código la relación con las fórmulas: el puntaje $QK^T / \sqrt{d_k}$ de la diapositiva 6 y la máscara de la diapositiva 8, que se aplica antes del softmax.

```
# Diap. 6: cada palabra atiende a las demás. score(q, k) = q k^T / sqrt(d_head).
scores = queries @ keys.transpose(-2, -1) / math.sqrt(self.d_head)
# Diap. 8: máscara causal ANTES del softmax. exp(-inf) = 0, así que el
# peso de atención hacia el futuro queda exactamente en 0.
scores = scores.masked_fill(self.causal_mask[:time, :time], float("-inf"))
weights = F.softmax(scores, dim=-1)
# Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V
attended = weights @ values
```

Antes de entrenar verificamos dos cosas con una secuencia de prueba de 16 caracteres: que los logits salieran con forma (1, 16, 65) y que la máscara causal funcionara. La máscara causal pone menos infinito en los puntajes de atención hacia posiciones futuras antes del softmax, así que su peso queda en 0 y cada carácter solo puede mirar a los anteriores (como leer un libro tapando con la mano todo lo que viene después). Comprobamos que la parte triangular superior de los mapas de atención fuera exactamente 0 en los tres bloques.

```
for weights in attention_maps:
    assert weights.shape == (1, N_HEADS, 16, 16)
    # Diap. 8 (S13): "cada token solo puede atender a los tokens anteriores".
    # Los pesos sobre la diagonal (j > i, el futuro) deben ser 0 tras el softmax.
    future = torch.triu(weights[0], diagonal=1)
    assert future.abs().max().item() == 0.0
```

Task 1.2: entrenamiento y perplejidad

Entrenamos con cross-entropy sobre todos los caracteres objetivo del batch, en CPU. La validación recorre de forma determinista todos los bloques completos del split, sin muestreo aleatorio, y calculamos la perplejidad como $\exp(L_val)$. La perplejidad se puede leer como el número de caracteres entre los que el modelo está dudando en promedio: un modelo que adivina al azar tendría 65.

Task 1.3: funciones de muestreo

Con el modelo entrenado, implementamos desde cero `sample_greedy`, `sample_top_k` y `sample_top_p`. Cada una recibe los logits crudos del último paso y devuelve solo el índice del siguiente carácter. Greedy toma siempre el argmax. Top-k divide los logits entre la temperatura τ , conserva los k mayores, aplica softmax y sortea entre ellos. Top-p (también llamado nucleus sampling) aplica temperatura y softmax, ordena las probabilidades y conserva el conjunto mínimo cuya masa acumulada alcanza p; luego renormaliza y sortea. La diferencia clave es que en top-k el número de candidatos es fijo, mientras que en top-p se adapta a qué tan segura está la distribución en cada paso.

La temperatura aparece en $P(w_i | ctx) = \exp(z_i / \tau) / \sum_j \exp(z_j / \tau)$. Un τ menor que 1 amplifica las diferencias entre logits y concentra la probabilidad en pocos caracteres; un τ de 1 deja la distribución como la aprendió el modelo.

En top-k se ve directamente la fórmula de la diapositiva 10: primero dividimos los logits entre τ y después dejamos en menos infinito todo lo que no está entre los k mayores, para que el softmax les asigne probabilidad 0.

```
# Diap. 10: P(siguiete token | contexto) = softmax(z / tau).
scaled = logits / tau
# Top-k: "solo considerar los k tokens más probables". El resto queda en -inf
# y el softmax les asigna 0, renormalizando entre los k que quedan.
top_values, top_indices = torch.topk(scaled, k)
filtered_logits = torch.full_like(scaled, float("-inf"))
filtered_logits.scatter_(0, top_indices, top_values)
probabilities = F.softmax(filtered_logits, dim=-1)
```

Task 2.1: generación de las 25 muestras

Escribimos la función `generate(prompt, max_new_tokens, strategy, **kwargs)` con la interfaz pedida y generamos cinco muestras de 200 caracteres para cada una de las cinco configuraciones de la Tabla 2. Usamos el prompt `'ROMEO: '` con un espacio final en lugar de `"ROMEO:"`; la diferencia es solo ese espacio en el contexto inicial. Cada muestra usa una semilla distinta y reproducible.

Tabla 2. Configuraciones de muestreo del Task 2.

Configuración	Estrategia	Parámetros
A	greedy	ninguno
B	top-k	$k = 5, \tau = 0.7$
C	top-k	$k = 50, \tau = 1.0$

Configuración	Estrategia	Parámetros
D	top-p	$p = 0.70, \tau = 0.7$
E	top-p	$p = 0.95, \tau = 1.0$

Además de leer las muestras, calculamos para cada configuración la tasa de repetición de 4-gramas (qué fracción de secuencias de cuatro caracteres se repite), la entropía de la distribución, la probabilidad máxima y el número de candidatos en cada paso. Estas métricas apoyan el análisis, pero no sustituyen citar ejemplos concretos de las muestras.

Task 3.1: carga de BERT

Para la entrega final cargamos `bert-base-multilingual-cased` desde el Hub de Hugging Face (un repositorio público de modelos, algo así como un GitHub de modelos de IA) con la librería `transformers`. La función `from_pretrained` descarga la configuración, los pesos y el vocabulario la primera vez y después los lee de la caché local, así que no necesitamos ningún script de descarga aparte. Llamamos a `model.eval()` para desactivar el dropout y que el mismo texto produjera siempre el mismo embedding.

Antes de extraer nada quisimos confirmar que cargamos el modelo correcto y no confiar solo en el nombre. El modelo resultó ser un `BertModel` con 12 capas, 12 cabezas de atención, dimensión oculta de 768, un vocabulario de 119,547 tokens (el BERT en inglés tiene unos 30 mil, así que el tamaño confirma que es el multilingüe) y 177,853,440 parámetros. También registramos el commit del repositorio descargado, `3f076fdb1ab68d5b2880cb87a0886f315b8146f8`, para que el experimento sea reproducible. Nuestro primer intento de leer ese commit desde la configuración del modelo devolvió "no disponible" en esta versión de `transformers`, así que lo tomamos del nombre de la carpeta de la caché, que guarda cada versión en `snapshots/<commit>/`.

Al cargar apareció un reporte que marcaba como UNEXPECTED los pesos `cls.predictions.*` y `cls.seq_relationship.*`. Al principio parecía un error, pero son las cabezas de preentrenamiento de BERT (la que adivina palabras tapadas con `[MASK]` y la que decide si una oración sigue a otra). `AutoModel` carga solo el encoder, que es lo que produce los embeddings, así que es correcto que esas cabezas se descarten.

Task 3.1: tokenización y ubicación de la palabra objetivo

BERT no recibe palabras sino tokens WordPiece: una palabra frecuente es un solo token y una poco frecuente se parte en fragmentos que empiezan con ##. El tokenizer además agrega [CLS] al inicio y [SEP] al final. Para ubicar la palabra objetivo buscamos su posición en caracteres dentro de la oración como palabra completa, para que vela no coincidiera dentro de otra palabra, y elegimos los tokens cuyo rango de caracteres (el `offset_mapping` que devuelve el tokenizer) cae dentro de ella. Como [CLS] tiene un rango vacío, nunca se elige, que es justo lo que pedía el enunciado.

En las siete oraciones la palabra objetivo quedó como un solo token. Las palabras de alrededor sí se partieron, por ejemplo vele ##ro, apa ##gón y ti ##bur ##ón. Revisamos el vocabulario y encontramos que banco, copa y vela tienen su propio token, mientras que apagón o velero no. Con 119,547 tokens repartidos entre 104 idiomas, solo las cadenas más frecuentes ganan un lugar propio; vela, además, se escribe igual en portugués, italiano y catalán.

Task 3.1: extracción de los embeddings

Pasamos cada oración completa por BERT dentro de `torch.no_grad()`, porque solo hacíamos inferencia. La salida `last_hidden_state` tiene forma (1, tokens, 768): un vector de la última capa por cada token. Tomamos el vector en la posición de la palabra objetivo; si hubiera estado partida, habríamos promediado sus fragmentos.

```
@torch.no_grad()
def target_embedding(sentence, word):
    encoding, positions = target_token_positions(sentence, word)
    # Diap. 6 y 7: sin máscara causal, el token de la palabra atiende a TODOS los
    # demás en las 12 capas, así que su vector final depende del contexto.
    outputs = bert(**encoding)
    # last_hidden_state: (1, tokens, 768) -> vector de la palabra objetivo (768,)
    return outputs.last_hidden_state[0, positions].mean(dim=0)
```

Obtuvimos una matriz de 7 por 768. Lo primero que notamos fue que las tres instancias de banco entran a BERT con el mismo id de token (55379), y lo mismo pasa con copa (39313) y vela (89018), pero salen con vectores distintos. Por ejemplo, la primera componente de banco vale -0.566 en el contexto financiero, 0.149 en el del parque y 0.005 en el de los peces. La entrada es idéntica y la diferencia la produce el contexto a través de las 12 capas de self-attention.

Task 3.2: similitud coseno, Word2Vec y PCA

Para medir cuánto difieren las instancias implementamos la similitud coseno con la fórmula del enunciado, $\text{sim}(u, v) = u^T v / (\|u\| \cdot \|v\|)$, y la verificamos contra `torch.nn.functional.cosine_similarity`. Calculamos todos los pares de la misma palabra: tres para banco, uno para copa y uno para vela.

Para la comparación estática usamos `word2vec-google-news-300`, el modelo Word2Vec preentrenado que distribuye `gensim`, con 3,000,000 de palabras y vectores de 300 dimensiones. Es el único Word2Vec que `gensim` descarga directamente, pesa unos 1.6 GB y se entrenó con noticias en inglés. Antes de comparar revisamos si nuestras palabras estaban en su vocabulario y cuáles eran sus vecinos más cercanos, para saber qué representaba cada vector.

Por último aplicamos PCA (una técnica que busca las direcciones en las que los datos varían más y proyecta los puntos sobre ellas, como encontrar el mejor ángulo para fotografiar un objeto en 3D sin perder su forma) para llevar los siete vectores de 768 dimensiones a 2. PCA no recibe las etiquetas de significado; esas solo las usamos después para colorear la gráfica.

Relación entre las fórmulas de las diapositivas y el código

Comentamos todo el código, tanto `lab8_minigpt.py` como las celdas de los dos notebooks, indicando qué fórmula o concepto de las diapositivas de la Semana 13 (NLP y Generación de Texto) implementa cada parte. La Tabla 3 resume esa correspondencia. Las fórmulas de LayerNorm y del feed-forward no aparecen en estas diapositivas; vienen del Proyecto 1 y así quedaron referenciadas en los comentarios. La similitud coseno viene del enunciado del laboratorio.

Tabla 3. Fórmulas y conceptos de las diapositivas de S13 y dónde quedaron implementados.

Diapositiva	Fórmula o concepto	Implementación en el código
3	Del texto a los números	Índices por carácter (Mini-GPT) e <code>input_ids</code> de WordPiece (BERT)
4	Word2Vec: un vector denso por palabra; significado como distancia geométrica	<code>word2vec[word]</code> , <code>most_similar</code> , similitud coseno y proyección PCA a 2D
6	Self-attention: $\text{softmax}(QK^T / \sqrt{d_k}) V$, todas las posiciones en paralelo	<code>CausalSelfAttention.forward</code> y codificación posicional sinusoidal
7	BERT: solo encoder, atención bidireccional sin máscara causal	<code>AutoModel</code> con <code>bert.eval()</code> y extracción de <code>last_hidden_state</code>

Diapositiva	Fórmula o concepto	Implementación en el código
8	GPT: máscara causal y predecir el siguiente token dado el contexto	<code>causal_mask</code> con <code>masked_fill(-inf)</code> , $y = x$ desplazado, cross-entropy y generación autoregresiva
9	Preentrenar y adaptar	Carga de pesos preentrenados con <code>from_pretrained</code> , sin entrenamiento
10	$P(\text{siguiente token} \mid \text{contexto}) = \text{softmax}(z / \tau)$	Logits $z = h W_{lm} + b_{lm}$, <code>softmax(logits / tau)</code> y entropía
10	$\tau \rightarrow 0$: siempre el token más probable	<code>sample_greedy</code> (argmax)
10	Top-k y top-p (nucleus sampling)	<code>sample_top_k</code> (topk y -inf) y <code>sample_top_p</code> (suma acumulada hasta p)

4. Resultados

Entrenamiento del Mini-GPT

La Tabla 4 muestra la evolución de la pérdida y la perplejidad durante las 10 épocas.

Tabla 4. Pérdida de entrenamiento, pérdida de validación y perplejidad por época.

Época	Loss train	Loss val	Perplejidad val
1	3.4077	3.3513	28.540
2	3.1755	2.8134	16.667
3	2.6037	2.4949	12.121
4	2.4210	2.3755	10.757
5	2.3376	2.3171	10.146
6	2.2924	2.2826	9.802
7	2.2547	2.2557	9.542
8	2.2178	2.2189	9.197
9	2.1845	2.1830	8.873
10	2.1558	2.1570	8.645

En la Figura 1 se ve que la pérdida cae rápido en las primeras tres épocas y luego sigue bajando de forma más lenta. Las curvas de entrenamiento y validación terminan prácticamente encimadas (2.1558 contra 2.1570), así que no hay sobreajuste.

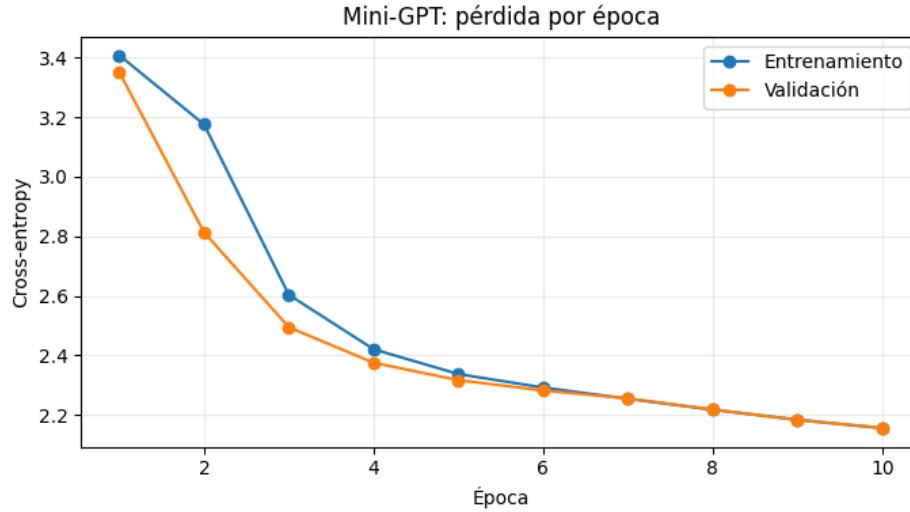


Figura 1. Pérdida de entrenamiento y validación del Mini-GPT por época.

La perplejidad final de validación fue 8.645, un poco por encima del rango esperado de 3.5 a 8.0. Como la curva todavía iba hacia abajo en la época 10, creemos que al modelo le faltó entrenamiento más que capacidad, pero lo reportamos tal como salió.

Estrategias de muestreo

La Tabla 5 resume las métricas promedio de las cinco muestras de cada configuración.

Tabla 5. Métricas promedio por configuración de muestreo.

Config.	Repetición 4-gramas	Entropía	Prob. máxima	Candidatos
A	0.954	2.126	0.382	1.00
B	0.186	1.780	0.459	5.00
C	0.022	2.255	0.339	50.00
D	0.224	1.807	0.450	3.99
E	0.027	2.238	0.343	15.02

Las cinco muestras de greedy (A-1 a A-5) salieron idénticas y cayeron en el mismo ciclo:

ROMEO: the the the the the the the the the the The the the the the the the the the the the the the

B conservó el formato de diálogo, con nombres de personaje como MORCUENE: y líneas como That and shis thand hous hat thell tharse. en B-2. C, en cambio, produjo palabras inventadas y símbolos como linkel'speerds&marak en C-1 y myirsAds en C-5. En E, para

la muestra E-1 identificamos los dos pasos de mayor y menor seguridad, que aparecen en la Tabla 6.

Tabla 6. Momentos de mayor y menor seguridad en la muestra E-1.

Tipo	Paso	Carácter elegido	Contexto previo	Prob. máxima	Candidatos
Seguro	195	salto de línea	...LABKET:	0.982	1
Seguro	75	espacio	...Thy by reip,	0.976	1
Inseguro	1	s	ROMEO:	0.131	35
Inseguro	199	f	...LABKET: [salto] Lo	0.110	37

Similitud coseno: BERT contra Word2Vec

Al revisar Word2Vec encontramos que las tres palabras sí están en su vocabulario, pero son poco frecuentes y sus vecinos más cercanos no tienen relación con su significado (Tabla 7).

Tabla 7. Posición por frecuencia y vecinos más cercanos en Word2Vec Google News.

Palabra	Posición (de 3 millones)	Vecinos más cercanos
banco	929,039	arriba, forza, deus, apenas, próxima
copa	1,068,243	boca, diez, você, ojos, solamente
vela	894,302	roberto, feliz, eto, tú, clase

La Tabla 8 compara la similitud coseno de cada par en los dos modelos.

Tabla 8. Similitud coseno entre instancias de la misma palabra en contextos distintos.

Palabra	Contexto 1	Contexto 2	Coseno BERT	Coseno Word2Vec	Diferencia
banco	finanzas	asiento	0.5753	1.0	0.4247
banco	finanzas	peces	0.6603	1.0	0.3397
banco	asiento	peces	0.6496	1.0	0.3504
copa	trofeo	recipiente	0.5667	1.0	0.4333
vela	navegación	iluminación	0.7999	1.0	0.2001

Visualización con PCA

Las dos componentes principales explicaron el 35.0 % y el 23.5 % de la varianza, un 58.6 % en total. La Figura 2 muestra la proyección, con un color por significado y una forma de marcador por palabra.

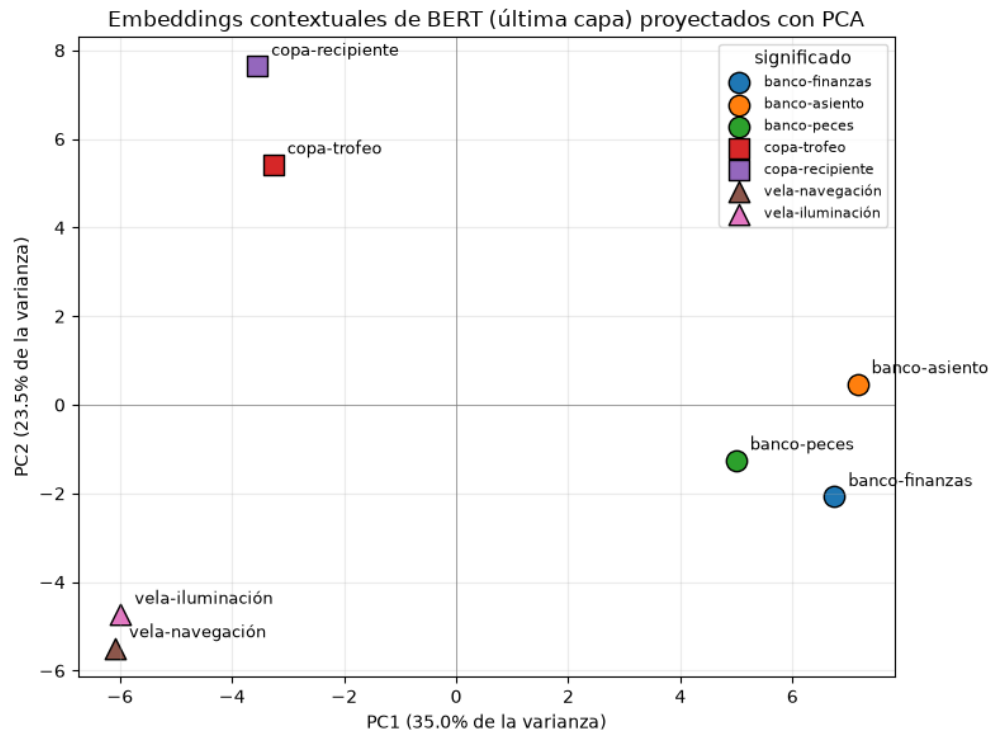


Figura 2. Embeddings contextuales de BERT (última capa) proyectados con PCA a dos dimensiones.

Se forman tres grupos lejanos entre sí: **banco** a la derecha (PC1 entre 5.0 y 7.2), **copa** arriba a la izquierda (PC2 entre 5.4 y 7.6) y **vela** abajo a la izquierda (PC2 entre -5.5 y -4.7). Entre grupos hay unas 12 unidades de distancia (12.5 entre **banco-finanzas** y **copa-trofeo**), mientras que dentro de cada grupo las distancias van de 0.77 a 2.77.

5. Discusión

Greedy, top-k y top-p

Greedy no tiene aleatoriedad, así que un mismo contexto produce siempre el mismo carácter, y por eso las cinco muestras A son idénticas. Después de escribir **the** , los últimos caracteres del contexto vuelven a ser los mismos, el argmax vuelve a elegir la misma continuación y el ciclo se cierra. Lo interesante es que la distribución aprendida no estaba concentrada (la probabilidad máxima promedio fue 0.382), pero greedy descarta el resto de la masa y siempre toma el máximo;

así la repetición de 4-gramas llegó a 0.954. Elegir lo más probable en cada paso no produce el texto más natural.

Entre B y C cambian dos cosas a la vez. Subir τ de 0.7 a 1.0 aplanó la distribución: la entropía media subió de 1.780 a 2.255 y la probabilidad máxima bajó de 0.459 a 0.339. Esa entropía la medimos sobre la distribución con temperatura antes del corte, así que refleja el efecto de τ y no el de k . Subir k de 5 a 50 dejó muchas más alternativas en cada paso y la repetición cayó de 0.186 a 0.022. El resultado fue que B produjo el texto más coherente, aunque repetitivo, y C el más creativo pero menos legible.

Top-p mostró su ventaja en E: en lugar de un número fijo de candidatos, usó 15.02 en promedio, contra 5 fijos en B y 50 en C, y ese número cambió según la seguridad del modelo. En la Tabla 6 se ve que el modelo está muy seguro (un solo candidato) en posiciones fijadas por el formato del texto, como el salto de línea tras el nombre de un personaje o el espacio tras una coma, y muy inseguro (35 a 37 candidatos) al inicio de una palabra, donde muchas letras son posibles.

Embeddings contextuales contra estáticos

Word2Vec dio coseno 1.0 en los cinco pares porque asigna un solo vector a cada palabra: `word2vec[ 'banco' ]` es el mismo arreglo en las tres oraciones. El coseno de un vector consigo mismo es 1 por definición, así que este modelo no tiene forma de distinguir el banco financiero del banco del parque ni del de peces. Además, como se entrenó con noticias en inglés, sus vectores para estas palabras representan sobre todo "palabra de un texto en español, italiano o portugués", como muestran sus vecinos de la Tabla 7. Aun con un Word2Vec entrenado en español el coseno seguiría siendo 1.0, porque eso es una propiedad de cualquier embedding estático y no del idioma.

BERT dio entre 0.567 y 0.800, entre 0.20 y 0.43 por debajo de Word2Vec. Ningún par llegó a 1 aunque la entrada fuera el mismo token, así que el contexto movió la representación de cada instancia. `copa` tuvo la similitud más baja (0.567), seguida de `banco finanzas` contra `asiento` (0.575). Nos llamó la atención que en `banco` el par `finanzas` contra `peces` (0.660) saliera más parecido que `finanzas` contra `asiento`, aunque los tres significados son distintos. Las oraciones de `finanzas` y de `peces` empiezan igual (`El banco ...`) y la palabra queda en la misma posición, mientras que la del `parque` tiene otra estructura (`Me senté en el banco ...`). El embedding contextual refleja el significado, pero también la sintaxis y la posición. `vela` tuvo la similitud más

alta (0.800); una posible explicación es que ambas oraciones comparten la estructura *vela para* seguida de un infinitivo.

Estos valores hay que leerlos en términos relativos. Los embeddings de BERT tienden a ocupar una región estrecha del espacio (a esto se le llama anisotropía), así que incluso palabras sin relación suelen tener coseno positivo. Un 0.57 no significa "57 % del mismo significado"; lo que sí dice algo es que los pares no llegan a 1 y que difieren entre sí. Una limitación de nuestro experimento es que solo tiene pares con significados distintos. Para medir qué tan bien distingue BERT habría que agregar pares con el mismo significado y verificar que su coseno sea claramente más alto.

¿Se separan los significados en el PCA?

La respuesta es que sí, pero de forma moderada. La separación dominante en la Figura 2 es por palabra y no por significado: con siete puntos de tres palabras distintas, lo que más varía es qué palabra es, y PCA dedica sus dos ejes a eso. Dentro de cada palabra los contextos sí quedan en puntos distintos. *banco-finanzas* y *banco-peces* son el par más cercano de *banco* (1.94), lo que coincide con su coseno más alto; *trofeo* y *recipiente* están a 2.24; y las dos *vela* quedan casi encimadas (0.77), lo que coincide con su coseno de 0.800.

La proyección no reproduce todas las relaciones. Según el coseno, el par más distinto de *banco* es *finanzas* contra *asiento* (0.575), pero en el plano el más alejado es *asiento* contra *peces* (2.77 frente a 2.58). Es la pérdida de información esperable al pasar de 768 a 2 dimensiones con solo el 58.6 % de la varianza, así que basamos la conclusión cuantitativa en los cosenos y usamos el PCA como apoyo visual. En conjunto, BERT ubica cada instancia en un punto distinto según su contexto, a diferencia de Word2Vec, donde las tres serían un solo punto; pero los distintos sentidos de una palabra quedan más cerca entre sí que de cualquier otra palabra.

6. Preguntas de análisis (Task 4.1)

a. La máscara causal y el objetivo de Language Modeling

El objetivo de Language Modeling es aprender $P(x_{t+1} \mid x_1, \dots, x_t)$, es decir, predecir el siguiente token viendo solo los anteriores. En el Task 1 lo vimos de forma concreta: durante el entrenamiento el modelo recibe un bloque completo y predice todas las posiciones a la vez, y el objetivo y es la misma entrada x corrida una posición. La respuesta de cada posición ya está dentro

de la entrada, justo un lugar a la derecha. La máscara causal es lo único que impide que el modelo la vea.

Si GPT usara atención bidireccional durante el preentrenamiento, cada posición podría atender a la siguiente y copiar el token que tiene que predecir. El modelo aprendería algo trivial, "copia al vecino de la derecha", en lugar de aprender cómo funciona el idioma. La loss bajaría casi a 0 y la perplejidad casi a 1, también en validación, porque ahí el modelo también ve el texto completo. Las métricas dirían que el modelo es casi perfecto y sería falso. El problema aparecería al generar: en ese momento el futuro todavía no existe, así que lo único que aprendió ya no le sirve y el texto no tendría sentido. Por eso en el Task 1 verificamos que la parte triangular superior de la atención fuera exactamente 0: esa verificación garantiza que nuestra perplejidad de 8.645 mide una predicción real y no una copia.

BERT sí es bidireccional y no cae en esta trampa porque cambia la tarea. En lugar de ocultar el futuro en la atención, oculta palabras en la entrada con [MASK] y le pide al modelo adivinarlas, así que la respuesta nunca está en lo que ve. GPT oculta la respuesta en la atención y BERT en la entrada; son dos soluciones al mismo problema.

b. Preentrenar y adaptar: elegir entre GPT y BERT según la tarea

La tensión viene de la misma máscara. GPT puede generar porque cada token solo ve hacia atrás, pero eso le quita información para entender una palabra. BERT ve a ambos lados y entiende mejor el contexto, pero por eso mismo no puede generar de forma autoregresiva.

Nuestras oraciones del Task 3 lo muestran bien. En varias, la pista que resuelve la ambigüedad aparece después de la palabra: "El banco central", "El banco de peces", "una copa de vino". Si comparamos "El banco central anunció..." con "El banco de peces se movió...", al llegar a **banco** lo único que un modelo causal ha visto en ambas oraciones es "El". Como con la máscara causal la representación de una posición depende solo de los tokens anteriores, GPT produciría exactamente el mismo vector para las dos instancias, con coseno 1.0, igual que Word2Vec. Esto no lo medimos ejecutando un GPT, pero se deduce directamente de cómo funciona la máscara. BERT, en cambio, pudo ver "central" y "peces", y en ese par obtuvimos 0.660.

El paradigma de preentrenar y adaptar resuelve la tensión porque no exige un solo modelo que haga todo bien. La parte cara, aprender el idioma a partir de un corpus enorme, la hace alguien una

sola vez y publica los pesos. Nosotros solo elegimos el modelo que conviene y lo adaptamos. Para entender texto (desambiguar, clasificar, medir similitud, reconocer entidades) conviene BERT, y el Task 3 lo muestra: sin entrenar nada, cargamos el modelo y en unos minutos, en CPU, obtuvimos embeddings que distinguen los significados de nuestras tres palabras. Para generar texto conviene GPT, porque su máscara causal es lo que le permite producir un token tras otro.

La diferencia con entrenar desde cero la vivimos en este mismo laboratorio. Nuestro Mini-GPT tiene 609,985 parámetros, lo entrenamos 10 épocas con Tiny Shakespeare, terminó con una perplejidad de 8.645 y genera palabras inventadas. BERT tiene 177,853,440 parámetros que no tuvimos que entrenar. Adaptar puede significar usar los embeddings tal como salen, que es lo que hicimos; agregar una capa encima y ajustar el modelo con pocos datos etiquetados (fine-tuning); o, en el caso de GPT, darle instrucciones en el prompt. En todos los casos partimos de un modelo que ya sabe el idioma, y la máscara deja de ser una limitación para convertirse en un criterio para escoger.

7. Conclusiones

Entrenamos un Mini-GPT de 609,985 parámetros que aprendió el formato de las obras de Shakespeare (nombres de personaje, saltos de línea, puntuación) y llegó a una perplejidad de validación de 8.645. Quedó un poco por encima del rango esperado, pero sin sobreajuste y con la curva todavía bajando, así que lo primero que probaríamos para mejorarlo es entrenarlo más épocas.

La estrategia de muestreo pesa tanto como el modelo. Con los mismos pesos, greedy produjo un ciclo de `the the the` con 0.954 de repetición, top-k con $k = 5$ dio el texto más coherente y top-k con $k = 50$ el más creativo pero menos legible. Top-p fue el único que adaptó el número de candidatos a la seguridad del modelo, de 1 a 37 en la misma muestra.

Comprobamos que BERT produce embeddings contextuales: la misma palabra, con el mismo token de entrada, terminó con cosenos entre 0.567 y 0.800 según la oración, mientras que Word2Vec dio 1.0 en todos los pares porque solo tiene un vector por palabra. También aprendimos a no sobreinterpretar estos números: el contexto no captura solo el significado, sino también la estructura de la oración, como mostró el par de `banco` finanzas y peces.

El PCA mostró que en BERT la identidad de la palabra pesa más que su significado, y que una proyección con el 58.6 % de la varianza puede deformar algunas distancias. Por eso apoyamos las conclusiones en los cosenos y usamos la gráfica como ayuda visual.

Lo que conecta todo el laboratorio es la máscara de atención. La máscara causal es la que hace válido el entrenamiento de GPT y le permite generar, y su ausencia es la que le permite a BERT entender una palabra por lo que viene después. Con modelos preentrenados no hay que escoger una sola de esas capacidades: se elige el modelo según la tarea, sin entrenar ninguno desde cero.

8. Referencias

- Devlin, J., Chang, M.-W., Lee, K., y Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171-4186.
- Holtzman, A., Buys, J., Du, L., Forbes, M., y Choi, Y. (2020). The curious case of neural text degeneration. *International Conference on Learning Representations (ICLR)*.
- Karpathy, A. (2015). *char-rnn: Multi-layer recurrent neural networks for character-level language models* [Repositorio de GitHub, conjunto de datos Tiny Shakespeare]. <https://github.com/karpathy/char-rnn>
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., y Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems* 26.
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., y Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI.
- Řehůřek, R., y Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45-50.

Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems* 30.

Wolf, T., et al. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP 2020: System Demonstrations*, 38-45.